

CLAUDE.md — Tingle Tracker

This file defines session rules, conventions, and context for all Claude Code sessions on this project. Read this file at the start of every session before making any changes.

Project Overview

Tingle Tracker is a companion platform for ASMR content.

- **Creators** link YouTube videos, receive tingle heatmaps and LLM-analyzed trigger reports
- **Listeners** log tingle moments during playback, build personal trigger profiles, and discover new creators
- **Discovery** matches listeners to creators by trigger overlap

Deployed at: <https://tingle-tracker.vercel.app> (update this once live) Repo: <https://github.com/thatsmyboye/tingle-tracker>

Monorepo Structure

```
tingle-tracker/
├─ apps/
│   ├─ web/           # Next.js 14 (App Router)
│   └─ mobile/        # Expo (React Native)
├─ packages/
│   ├─ ui/            # Shared component library
│   ├─ database/       # Supabase client + generated types
│   ├─ ai/            # Claude API wrapper + prompt templates
│   └─ types/         # Shared TypeScript interfaces
├─ supabase/
│   └─ migrations/    # SQL migration files (never edit applied migrations)
├─ CLAUDE.md
├─ turbo.json
└─ package.json
```

Package Manager & Tooling Rules

- **Always use** `pnpm` — never `npm` or `yarn`. If a command fails, check for a pnpm equivalent.
 - Run scripts from the repo root using `pnpm --filter <package> <script>` or `turbo run <task>`
 - Install packages: `pnpm add <package> --filter <app-or-package>`
 - Install dev deps: `pnpm add -D <package> --filter <app-or-package>`
-

Tech Stack

Layer	Technology	Location
Web frontend	Next.js 14, App Router, TypeScript	apps/web
Mobile frontend	Expo SDK 51+, React Native	apps/mobile
Shared UI	Tailwind CSS + Radix UI (web), NativeWind (mobile)	packages/ui
Auth + DB	Supabase (Postgres + RLS + Realtime)	packages/database
Background jobs	Inngest	apps/web/inngest/
AI / LLM	Claude API via @anthropic-ai/sdk	packages/ai
Billing	Stripe	apps/web/app/api/stripe/
Hosting	Vercel	—

Database Conventions

Supabase

- Client is initialized in `packages/database/src/client.ts`

- Always use the **generated TypeScript types**: `packages/database/src/types.ts`
- Regenerate types after any schema change:

```
pnpm supabase gen types typescript --project-id <PROJECT_ID> > packages/databa
```

- **Never edit applied migrations.** Create a new migration file for all schema changes:

```
pnpm supabase migration new <description>
```

- Migration files live in `supabase/migrations/` and are numbered sequentially

RLS Policies — Core Rules

- `tingle_events` : users can only SELECT/INSERT/UPDATE/DELETE their own rows
(`auth.uid() = user_id`)
- `user_profiles` : users can only SELECT/UPDATE their own row
- `creators` : publicly readable; write requires `is_creator = true` on the user's profile
- `content` : publicly readable; write requires ownership via `creators.user_id = auth.uid()`
- `insights_cache` : publicly readable (by content owner); write is service-role only (Inngest)
- `trigger_tags` : publicly readable; write is service-role only

Naming

- Tables: `snake_case` , plural (e.g., `tingle_events` , `trigger_tags`)
- Columns: `snake_case`
- Foreign keys: `<table_singular>_id` (e.g., `content_id` , `creator_id`)
- Timestamps: always `created_at` + `updated_at` with `DEFAULT now()`
- UUIDs: always use `gen_random_uuid()` as default for primary keys

AI / Claude API

- All Claude API calls go through `packages/ai/src/client.ts`

- Use `claude-sonnet-4-20250514` as the model — never hardcode model strings elsewhere
- Prompt templates live in `packages/ai/src/prompts/`
- Keep system prompts in separate `.ts` files, not inline in API routes
- AI calls from the web app happen exclusively via **Inngest jobs** (never in API routes directly, to avoid timeout issues)
- Always set a `max_tokens` budget appropriate to the task

Inngest Jobs (Background)

All Inngest functions live in `apps/web/inngest/functions/`. Naming convention: `<noun>.<verb>.ts`

Key functions:

- `content.analyze.ts` — triggered when a creator links a video; calls Claude to classify trigger tags
 - `insights.generate.ts` — runs on a schedule (or on demand) to produce creator insight reports
 - `tingle.aggregate.ts` — aggregates raw `tingle_events` into heatmap-ready data
-

YouTube Integration

- Use the **YouTube Data API v3** for video metadata (title, description, duration, thumbnails)
 - Use **YouTube oEmbed** for lightweight embeds where API quota is a concern
 - Use **YouTube Captions API** (if transcript available) to feed Claude trigger classification
 - Store `youtube_video_id` on the `content` table — never store full YouTube URLs
 - The web player wraps `react-youtube` or a custom `iframe` embed component in `packages/ui`
 - **Tingle logging** uses the YouTube IFrame Player API to access `getCurrentTime()` on tap/click
-

Tingle Logging — UX Rules

- Mobile is the primary surface (ASMR is headphone/phone-native)
 - A single **tap** logs a tingle event at the current playback timestamp
 - A **long press** (300ms) opens an intensity picker (1–5 scale: • • • • •)
 - Events are debounced: minimum 500ms between events (prevents accidental double-logging)
 - Events are queued locally (AsyncStorage on mobile, sessionStorage on web) if offline, then flushed on reconnect
 - Timestamp is stored as `timestamp_ms` (integer, milliseconds from video start)
-

Trigger Taxonomy

Categories: `visual` | `aural` | `tactile_adjacent`

The `trigger_tags` table is seeded via `supabase/seed.sql` . Never add tags programmatically from user input — tags are curated. LLM classification maps content to existing tags only.

Environment Variables

Web (`apps/web/.env.local`)

```
NEXT_PUBLIC_SUPABASE_URL=  
NEXT_PUBLIC_SUPABASE_ANON_KEY=  
SUPABASE_SERVICE_ROLE_KEY=  
ANTHROPIC_API_KEY=  
YOUTUBE_DATA_API_KEY=  
INNGEST_EVENT_KEY=  
INNGEST_SIGNING_KEY=  
STRIPE_SECRET_KEY=  
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=  
STRIPE_WEBHOOK_SECRET=
```

Mobile (`apps/mobile/.env`)

```
EXPO_PUBLIC_SUPABASE_URL=  
EXPO_PUBLIC_SUPABASE_ANON_KEY=  
EXPO_PUBLIC_API_BASE_URL=
```

Never commit `.env` files. Never log environment variables.

Code Style

- TypeScript strict mode everywhere — no `any` types without an explicit comment explaining why
- Use `zod` for runtime validation of API inputs and external API responses
- Prefer `async/await` over `.then()` chains
- Error handling: always wrap external API calls (YouTube, Claude, Stripe) in `try/catch` and return typed error objects
- Components: functional only, no class components
- Exports: named exports preferred; default exports only for Next.js pages/layouts and Expo screens

File Naming

- Components: `PascalCase.tsx`
 - Utilities / hooks: `camelCase.ts`
 - API routes (Next.js): `route.ts` inside the appropriate `app/api/` folder
 - Inngest functions: `<noun>.<verb>.ts`
-

Epics & Story Points Reference

Epic ID	Label	Phase	Points
<code>data-model</code>	Data Model	1	8
<code>content-ingest</code>	Content Ingest	1	5
<code>tingle-logger</code>	Tingle Logger	1	8

heatmap-viz	Heatmap Viz	1	5
trigger-taxonomy	Trigger Taxonomy	2	3
llm-pipeline	LLM Pipeline	2	13
creator-dashboard	Creator Dashboard	2	8
user-profile	User Trigger Profile	3	8
discovery	Discovery Engine	3	13

Tag your commits with the epic ID: `git commit -m "[tingle-logger] add intensity picker component"`

Session Startup Checklist

Before starting work each session:

1. Read this file fully
 2. Check `git status` — confirm you are on the correct branch
 3. Run `pnpm install` if `package.json` has changed
 4. Run `pnpm supabase status` to confirm local DB is running (if working on DB)
 5. Confirm which epic you are working on and note its acceptance criteria
-

What NOT to Do

- Do not install packages without checking if they already exist in the monorepo
- Do not write raw SQL in application code — use the Supabase client
- Do not call the Claude API from Next.js API routes directly — route through Inngest
- Do not store YouTube URLs — store `youtube_video_id` only
- Do not use `any` without a comment
- Do not commit secrets or `.env` files
- Do not edit applied Supabase migrations

